LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Smart Thermostat Controller | Hypothesis-based testing

Student	Dinesh
SRN	PES1UG24CS157
Assigned repository	SETAPESU26/L6_L7_29_thermostat_control
Application	Thermostat control

1. Objective

The purpose of this lab was to use Hypothesis-based fuzz testing to identify input-validation problems in the assigned thermostat-control application. The tests were executed against the original implementation, the defects were addressed, and the same test suite was run again against the corrected implementation.

2. Defects found in the original implementation

Function	Defects demonstrated by the baseline run
parse_setpoint	Insufficient input/type and format validation. None caused TypeError; malformed values such as 21X could be accepted; non-finite values such as nanC and infF were not rejected.
average_temperature	Insufficient input/type and finite-number validation. None caused AttributeError; NaN and infinity could be accepted as readings.
schedule_slot	Did not enforce strict HH:MM format or valid clock ranges. Values such as 24:00, 12:60, -1:30 and 7:3 were accepted. The original output also retained inconsistent minute formatting.

3. Baseline fuzz-test results

The original implementation was tested using the 31-test suite. The recorded run reported **21 failed and 10 passed** in **19.72 seconds**. Hypothesis produced minimized counterexamples including None, malformed setpoint strings, NaN readings, and out-of-range time values.

Figure 1. Baseline pytest summary captured from the user's PowerShell session.

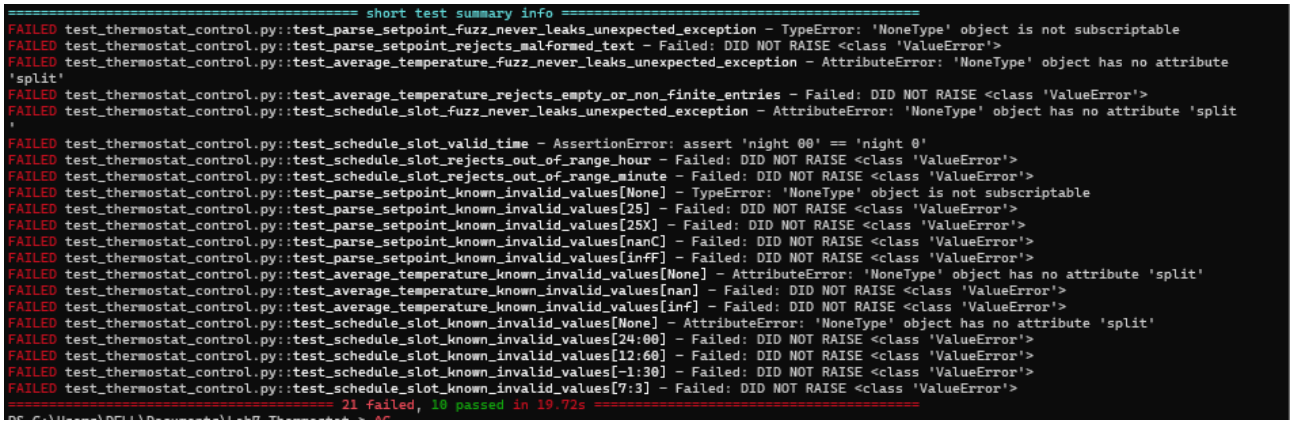
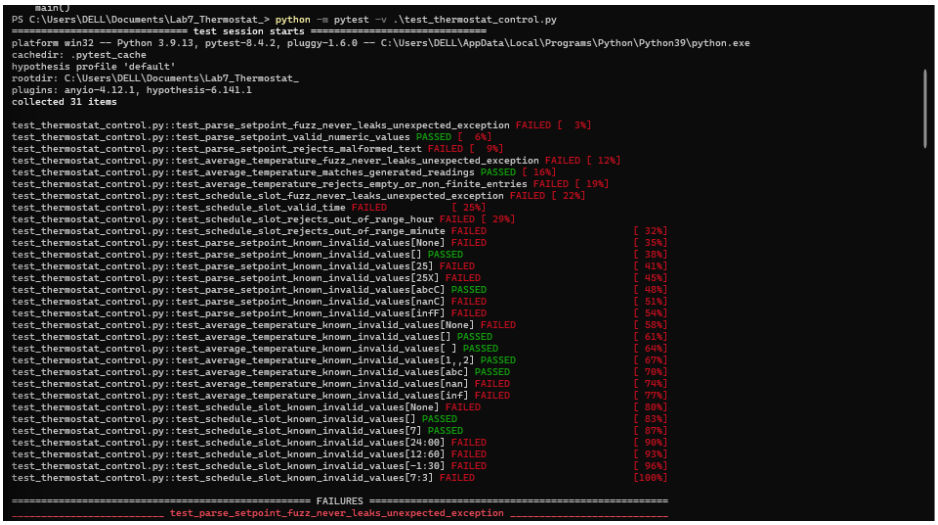


Figure 2. Baseline test progress and individual failing test names.


main()
PS C:\Users\DELL\Documents\Lab7_Thermostat_> python -m pytest -v .\test_thermostat_control.py
============================= test session starts =============================
platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\Python39\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\DELL\Documents\Lab7_Thermostat_
plugins: anyio-4.12.1, hypothesis-6.141.1
collected 31 items
test_thermostat_control.py::test_parse_setpoint_fuzz_never_leaks_unexpected_exception FAILED [3%]
test_thermostat_control.py::test_parse_setpoint_valid_numeric_values PASSED [6%]
test_thermostat_control.py::test_parse_setpoint_rejects_malformed_text FAILED [9%]
test_thermostat_control.py::test_average_temperature_fuzz_never_leaks_unexpected_exception FAILED [12%]
test_thermostat_control.py::test_average_temperature_matches_generated_readings PASSED [16%]
test_thermostat_control.py::test_average_temperature_rejects_empty_or_non_finite_entries FAILED [19%]
test_thermostat_control.py::test_schedule_slot_fuzz_never_leaks_unexpected_exception FAILED [22%]
test_thermostat_control.py::test_schedule_slot_valid_time FAILED [25%]
test_thermostat_control.py::test_schedule_slot_rejects_out_of_range_hour FAILED [29%]
test_thermostat_control.py::test_schedule_slot_rejects_out_of_range_minute FAILED [32%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[None] FAILED [35%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[] PASSED [38%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[25] FAILED [41%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[25X] FAILED [45%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[abcC] PASSED [48%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[nanC] FAILED [51%]
test_thermostat_control.py::test_parse_setpoint_known_invalid_values[infF] FAILED [54%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[None] FAILED [58%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[] PASSED [61%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[ ] PASSED [64%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[1,,2] PASSED [67%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[abc] PASSED [70%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[nan] FAILED [74%]
test_thermostat_control.py::test_average_temperature_known_invalid_values[inf] FAILED [77%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[None] FAILED [80%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[] PASSED [83%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[7] PASSED [87%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[24:00] FAILED [90%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[12:60] FAILED [93%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[-1:30] FAILED [96%]
test_thermostat_control.py::test_schedule_slot_known_invalid_values[7:3] FAILED [100%]
================================== FAILURES ===================================
_____ test_parse_setpoint_fuzz_never_leaks_unexpected_exception _____

4. Corrections implemented

parse_setpoint: Validates that the input is a string, trims surrounding whitespace, requires a numeric value followed by C or F, rejects non-finite values, converts Fahrenheit to Celsius, and rounds the result to one decimal place.

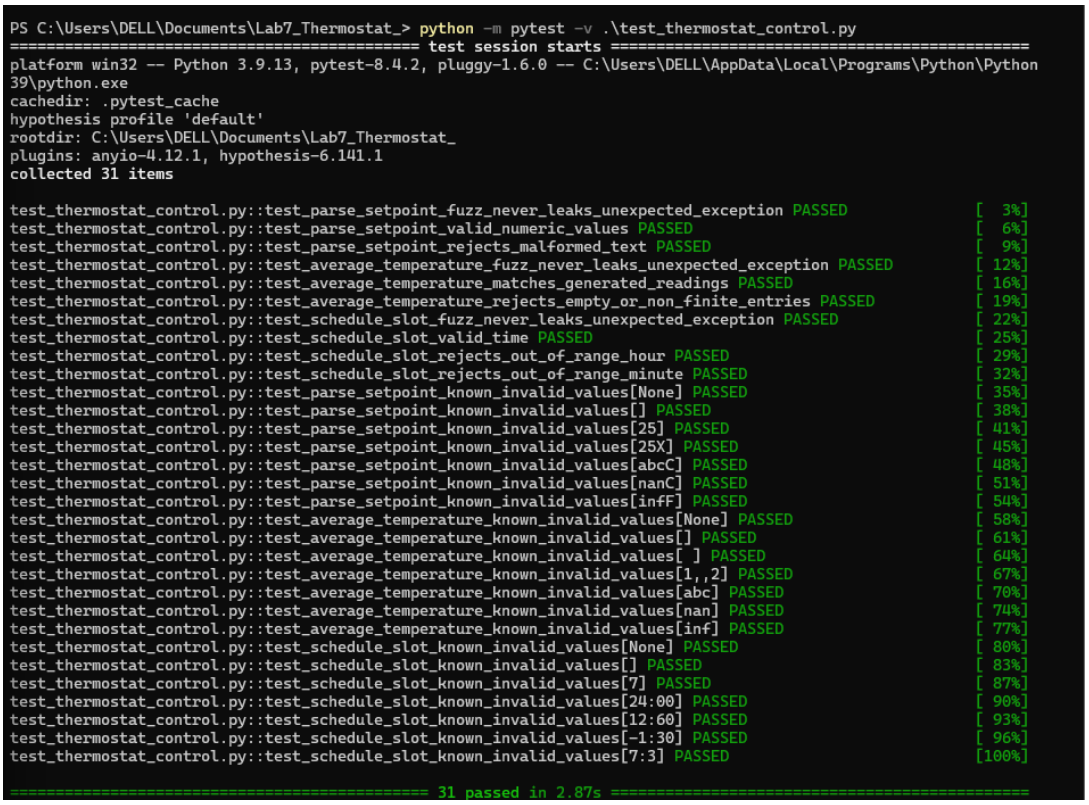
average_temperature: Validates the input type and rejects empty CSV entries. It converts each entry to a number, rejects non-finite readings, and computes the mean while reducing overflow risk by dividing readings before summing.

schedule_slot: Requires the HH:MM format, checks that hours are between 00 and 23 and minutes between 00 and 59, then classifies times before 06:00 as night and the remaining valid times as day.

5. Fixed implementation test results

After the validation fixes were applied, the same 31 tests were run again. The recorded result was **31 passed, 0 failed** in **2.87 seconds**.

Figure 3. Complete passing pytest run captured from the user's PowerShell session.



6. Test coverage

The test module includes Hypothesis-generated arbitrary text and None values; generated finite numeric setpoints with C/F units; generated lists of finite temperature readings; generated valid hours and minutes; malformed-input checks; and out-of-range hour/minute checks. Parameterized regression cases cover missing values, empty values, unsupported suffixes, NaN/Infinity, and invalid clock inputs.

7. Supplementary evidence and environment

The supplied **baseline_reproduction.txt** documents representative original-code behaviours, including TypeError for parse_setpoint(None), acceptance of a malformed setpoint, NaN output from average_temperature, and acceptance of invalid clock values. The supplied **fixed_smoke_tests.txt** records successful valid-input and invalid-input checks after the fixes.

The recorded environment was Python 3.9.13, pytest 8.4.2, and Hypothesis 6.141.1. The test suite can be run from the lab folder with:

```
python -m pip install hypothesis pytest
python -m pytest -v test_thermostat_control.py
```

8. Submission files

The core submission files are **thermostat_control.py** (corrected application), **test_thermostat_control.py** (updated Hypothesis tests), and this PDF report. **thermostat_control_fixed.py** is a backup copy of the corrected application.

Evidence scope: This report includes the supplied baseline and fixed pytest screenshots, plus the supplied reproduction and smoke-test text. It does not claim any additional interactive application screenshots were captured.